



Alma Mater Studiorum-Università di Bologna
Scuola di Ingegneria

Functional Programming & Stream di operazioni in Java

Corso di Laurea in Ingegneria Informatica
Anno accademico 2023/2024

Prof. ENRICO DENTI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)

DAL «COME» AL «COSA»

- Nell'approccio tradizionale, specificare una operazione significa tipicamente dire *come farla istruzione per istruzione*
 - si eseguono le singole sotto-operazioni immediatamente
 - l'ordine delle sotto-operazioni è quello con cui sono scritte
 - chiaro, ma rende di fatto impossibile cambiare successivamente *come* fare una cosa senza riscrivere il codice
 - in particolare, rende impossibile parallelizzarlo senza rifarlo da zero
- Ciò è tipico del *paradigma imperativo*, in cui un programma è appunto una *sequenza di ordini*
 - ossessione del controllo: chi scrive il codice deve (vuole) controllare ogni singolo dettaglio di ogni singola operazione – non delega nulla
 - risultato: *si viene sopraffatti da tanto controllo* – difficoltà ad astrarre, impossibilità di prevedere tutto e pianificare tutto... ☹

DAL «COME» AL «COSA»

- Esistono però *altri paradigmi*, non orientati al controllo, in cui il focus è su *cosa si vuole*, lasciando sullo sfondo *come* farlo
 - il *paradigma dichiarativo* esprime gli *assiomi* e le *regole* che esprimono le *relazioni* vere nel dominio
 - il *paradigma funzionale* esprime la *trasformazione* dei dati in termini di *funzioni* che agiscono via via su di essi
- Come si computa in tali paradigmi?
 - nel *paradigma dichiarativo* si computa *cercando di raggiungere un obiettivo (goal)* espresso tipicamente da una domanda (query) tramite un processo di *inferenza logico-deduttiva*
 - nel *paradigma funzionale* si computa *fornendo i dati iniziali* e *prelevando dall'ultima funzione* i dati di uscita

ESEMPIO (1/3)

- Problema: data una lista di stringhe, produrre la lista delle *corrispondenti lunghezze*
 - ad esempio, data la lista ["pippo", "pluto", "paperino", "zio"], vogliamo ottenere in risposta la lista [5, 5, 8, 3]
- Paradigmi a confronto
 - *nel paradigma imperativo* ci si focalizza sulla *sequenza di ordini*
 - creare preventivamente una lista di interi per il risultato (vuota)
 - scorrere la lista iniziale elemento per elemento → ciclo for (o foreach)
 - per ogni elemento, calcolare la corrispondente lunghezza (un intero)
 - inserire tale intero nella nuova lista di destinazione

```
List<String> source = List.of("pippo", "pluto", "paperino", "zio");  
List<Integer> result = new ArrayList<>();  
for (String s: source) result.add(s.length());  
System.out.println(result);
```

Controllo!

ESEMPIO (2/3)

- Problema: data una lista di stringhe, produrre la lista delle *corrispondenti lunghezze*
 - ad esempio, data la lista ["pippo", "pluto", "paperino", "zio"], vogliamo ottenere in risposta la lista [5, 5, 8, 3]
- Paradigmi a confronto
 - *nel paradigma dichiarativo* si esprimono i *fatti* e le *relazioni fra essi*
 - a lista iniziale vuota corrisponde lista di uscita vuota
 - alla lista di stringhe [Testa|Coda] corrisponde la lista di interi [TestaN |CodaN]
 - TestaN è la lunghezza della stringa Testa
 - CodaN è la (sotto)lista delle lunghezze relative alla (sotto)lista Coda

Ad esempio, in Prolog:

```
list_len([], []).
```

```
list_len([H|T], [HN|TN]) :- list_len(T, TN), atom_length(H, HN).
```

Non c'è il controllo: solo *le relazioni vere* nel dominio

```
Query: list_len([pippo,pluto,paperino,zio], L).
```

```
Reply: L / [5,5,8,3]
```

ESEMPIO (3/3)

- Problema: data una lista di stringhe, produrre la lista delle *corrispondenti lunghezze*
 - ad esempio, data la lista ["pippo", "pluto", "paperino", "zio"], vogliamo ottenere in risposta la lista [5, 5, 8, 3]
- Paradigmi a confronto
 - *nel paradigma funzionale* si esprimono le *trasformazioni dei dati*
 - detta S la sequenza dei dati iniziali
 - ogni elemento va *mappato* nella sua lunghezza: $element \rightarrow length(element)$
 - la sequenza S' dei dati così trasformati costituisce il risultato

Non c'è il controllo: solo *come trasformare i dati*

Schema generale:

["pippo", "pluto", "paperino", "zio"] $\xrightarrow{e \rightarrow length(e)}$ [5, 5, 8, 3]
`List<Integer> res = lista.trasforma(s->s.length()).recupera(...);`

PARADIGMA DICHIARATIVO E INVERTIBILITÀ

- A differenza del paradigma imperativo (e funzionale), il **paradigma dichiarativo è intrinsecamente invertibile**
 - essendo basato solo sulle *relazioni* fra i dati, non prestabilisce quali siano gli input e quali gli output: **dipende dalla singola query**
- Esempio: una funzione per calcolare la somma
 - nel paradigma imperativo, la funzione `sum(int x, int y)` ha `x, y` come ingressi e la somma come valore restituito: *non è possibile usarla «a rovescio»*, fornendo `x` e la somma per ottenere `y`
 - nel **paradigma dichiarativo invece sì** 😊

```
sum(X, 0, X).  
sum(X, s(Y), s(Z)) :- sum(X, Y, Z).
```

- 1) se sommi X a 0, otterrai ancora X
- 2) se sommi X al successore di Y, otterrai il successore di Z (essendo Z a sua volta il risultato della somma di X e Y)

```
sum(s(s(0)), s(s(s(0))), N).  
N = s(s(s(s(s(0)))))  
sum(s(s(0)), Z, s(s(s(0)))).  
Z = s(0)
```

sum(2,3,N) → N=5

sum(2,Z,3) → Z=1

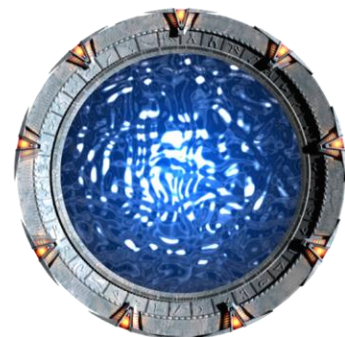


LINGUAGGI MULTI-PARADIGMA

- Può essere molto interessante *far coesistere nello stesso linguaggio paradigmi di computazione diversi*
 - possibilità di sfruttare il più adatto a ogni circostanza
 - concatenare stringhe → Imperativo
 - calcolare le derivate simboliche → Dichiarativo
 - trovare la stringa più corta in una lista → Funzionale
- Ma coniugare più paradigmi non è cosa semplice
 - occorre progettare molto bene, fin dall'inizio, le API
 - ma soprattutto, occorre un linguaggio «intrinsecamente blended», progettato per essere tale fin dalle fondamenta
- Scala e Kotlin lo sono, Java no
 - Java: troppa base installata per cambiamenti profondi
 - Scala, Kotlin: blend di paradigma imperativo + a oggetti + funzionale

JAVA COME LINGUAGGIO MULTI-PARADIGMA

- Per far coesistere *in un linguaggio già esistente* paradigmi di computazione diversi occorre un approccio non invasivo:
 - non si può «rifare tutto»: il know how e la base installata devono essere preservati
 - necessità di *evitare interferenze reciproche* che possano minare alla base il modello (e la correttezza) di funzionamento
- **IDEA: incapsulare il nuovo paradigma in una astrazione ad hoc che funga da «stargate»**
 - una porta verso un «universo parallelo» con altre leggi
→ nel caso OOP, un *oggetto* ad hoc
 - esternamente, un oggetto come un altro, coi suoi metodi
→ usabile secondo il classico paradigma imperativo OOP
 - internamente, un *motore* che opera secondo un *paradigma alternativo*
→ computa secondo sue leggi, nel suo «universo alternativo»





FUNCTIONAL PROGRAMMING IN JAVA: STREAM DI OPERAZIONI

- Gli **stream di operazioni** sono l'astrazione introdotta da Java per **incapsulare il paradigma di programmazione *funzionale***
 - o almeno parte di esso..
- Ispirati al functional programming, gli stream di operazioni promuovono **uno stile in cui ragionare *non più* in termini di *controllo*, ma di *come trasformare i dati* via via**
 - richiede un pari cambio di *atteggiamento mentale*
- Focus su ***cosa si vuole***, lasciando sullo sfondo come farlo

NB: Scala e Kotlin incorporano tale paradigma fin dalle fondamenta, quindi l'astrazione stream non è necessaria



IL FOCUS:

LAVORARE SU INSIEMI DI ELEMENTI

- Tipicamente, un algoritmo elabora un insieme di elementi memorizzati «da qualche parte», in qualche *collection*
- L'algoritmo stabilisce cosa si debba fare *per ogni elemento*
 - **tenerlo o scartarlo**, in base a certe condizioni
 - **trasformarlo** in qualcos'altro, o **usarlo** per fare qualcosa
- Alla fine, con gli elementi rimasti si sintetizza il «risultato»
 - si **accumulano gli elementi** in un'altra collection
 - oppure magari li si **conta, raggruppa**, se ne fa la **media**, etc.

TIPICO APPROCCIO IMPERATIVO

Ciclo su ogni elemento, e per ogni elemento:

- if, switch, trasformazioni,...
- assegnamenti a *variabili temporanee di appoggio* a ogni giro

Controllo!

*Se qualcosa va storto,
il debug... AUGURI !*

IL FOCUS: LAVORARE SU INSIEMI DI ELEMENTI

- Tipicamente, un algoritmo elabora un insieme di elementi memorizzati «da qualche parte», in qu
1^a FASE: estrarre iterativamente gli elementi
- L'algoritmo stabilisce cosa si debba fare *per ogni elemento*
 - tenerlo o scartarlo, in base a certe condizioni
 - trasformarlo in qualcos'altro, o usarlo per fare qualcosa
2^a FASE: operare sul singolo elemento
- Alla fine, con gli elementi rimasti si sintetizza il «risultato»
 - si accumulano gli elementi in un'altra collect
3^a FASE: sintesi del risultato finale
 - oppure magari li si conta, raggruppa, se ne fa la media, etc.

TIPICO APPROCCIO IMPERATIVO

Ciclo su ogni elemento, e per ogni elemento:

- if, switch, trasformazioni,...
- assegnamenti a *variabili temporanee di appoggio* a ogni giro

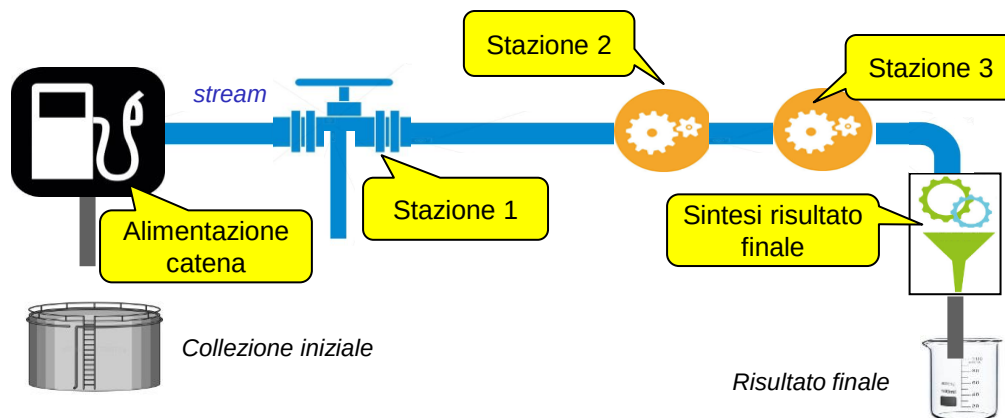


LAVORARE SU INSIEMI DI ELEMENTI CON UN APPROCCIO FUNZIONALE

- L'approccio funzionale mantiene queste tre fasi, ma consente di riorganizzarle in uno schema più pratico e modulare: la «catena di montaggio»
- Vantaggi
 - si **automatizza l'estrazione degli elementi** dalla collection (fase 1), *evitando l'iterazione esplicita e le relative variabili di appoggio*
 - si **evidenziano e linearizzano le «tappe»** della trasformazione (fase 2), *senza necessità di introdurre variabili di appoggio e assegnamenti*
 - si **cattura efficacemente la sintesi finale** del risultato (fase 3) come *ultimo stadio della catena*
- **Focus sul «cosa» si deve fare anziché sui meccanismi** per farlo → espressività, chiarezza, debug semplificato

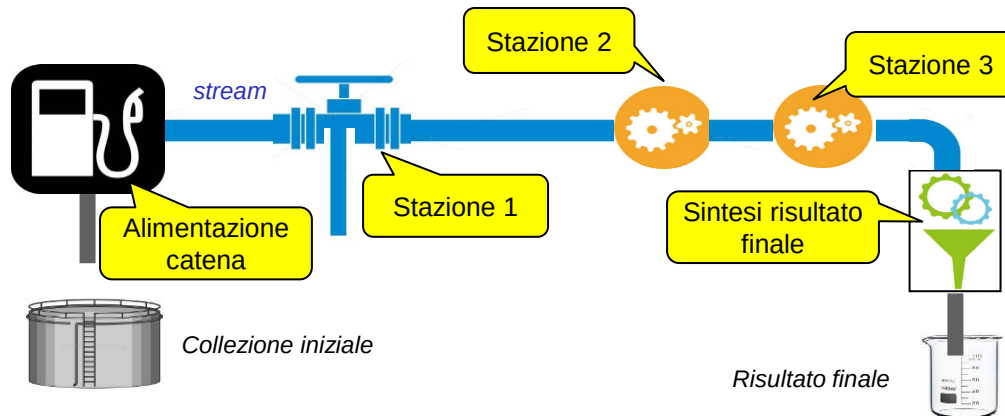
STREAM COME CATENA DI MONTAGGIO

- Lo stream è sostanzialmente **una catena di montaggio**, in cui:
 - all’inizio vi è una «pompa» che **alimenta** la catena, fornendole gli elementi su cui via via operare (sostituisce il classico «ciclo»)
 - in mezzo, ogni «stazione» della catena **elabora/filtra** ciò che le arriva: sono disponibili *molti tipi di «stazioni» già pronti*
 - alla fine, l’ultimo stadio sintetizza il **risultato finale**: sono disponibili *molti tipi di «stadi finali» già pronti*



STREAM: FASI FONDAMENTALI

- Ci sono dunque *tre momenti* = *tre categorie di funzionalità*:
 1. **creazione** dello stream → operazione di **creazione**
 2. **uso** dello stream → operazioni di **trasformazione**
 3. **sintesi** del risultato → operazione di **riduzione**
- Si computa *montando le «giuste» stazioni nel «giusto» ordine e scegliendo il riduttore finale «opportuno»*





STREAM: FASI FONDAMENTALI

- Fase 1: creazione

- produce uno stream *a partire da una sorgente esistente* (collection)
→ metodo factory **stream()** applicabile a qualunque collection
- in alternativa, può prevedere un *generatore* capace di *generare dinamicamente gli elementi* uno ad uno secondo una certa regola

- Fase 2: trasformazione

- ogni stazione svolge un'operazione: filtra, separa, modifica, etc.
→ tanti metodi disponibili: **filter**, **map**, etc.

- Fase 3: riduzione

- estrae e/o computa il «risultato finale»
→ metodo **collect** e altri
- fatta la riduzione, lo stream è chiuso e non può più essere usato

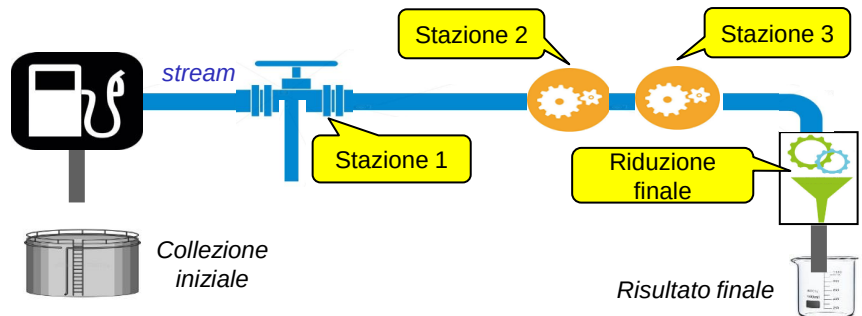


STREAM: FUNZIONAMENTO

- Uno stream, dunque, *non* è una collection
 - è un'infrastruttura di elaborazione, una catena di montaggio che «passa elementi da una stazione alla successiva»
- si basa sul pattern cascading → *fluent interface*
 - ogni operazione restituisce un nuovo stream in cui scorrono i dati «trasformati» dalla stazione precedente
- svolge le operazioni in modo *lazy*
 - non le esegue immediatamente, ma solo *quando* e *se* servono
 - le esegue *non necessariamente nell'ordine* in cui sono scritte
 - non facendo ipotesi sull'ordine, rende *facile parallelizzare* l'algoritmo
→ basta sostituire a uno stream un parallel stream: null'altro cambia!

ELABORARE CON GLI STREAM

- Trasformazioni tipiche
 - **filter**: fa passare solo gli elementi «tali che»...
 - **map**: trasforma tutti gli elementi in qualcos'altro (funzione $A \rightarrow B$)
 - **sorted**, **distinct**, **limit**, **skip**, ...
- Riduttori tipici
 - **collect**: accumula gli elementi in un *collettore*
 - collections, ma non solo...
 - **min**, **max**, **count**
 - **forEach**, **forEachOrdered**
 - **findFirst**, **findAny**
 - **allMatch**, **anyMatch**, **noneMatch**





ELABORARE CON GLI STREAM: LA FASE FINALE

- Al termine della catena, il *riduttore* sintetizza il risultato
- Il riduttore più usato, **collect**, accumula in un *collettore*
- Ci sono fondamentalmente *due categorie* di collettori:
 - quelli che semplicemente **accumulano** gli elementi in una *collection*
 - quelli che invece **elaborano gli elementi** per sintetizzare il risultato
- Collettori disponibili nella libreria *Collectors*
 - collettori che accumulano:
toList, **toSet**, **toMap**, **toArray**, **toCollection**, ...
 - collettori che sintetizzano:
counting, **averaging***, **summing**, **summarizing***, **maxBy**,
minBy, **joining**, **groupingBy***, **partitioningBy***, ...

Pensare con gli stream



THINK FUNCTIONALLY

PENSARE CON GLI STREAM

- Costituendo un totale cambio di paradigma, gli stream richiedono un pari **cambio di atteggiamento mentale**
 - occorre pensare *non più* in termini di controllo e «variabili»
 - ma in termini di **come trasformare i dati via via**
- Il paradigma imperativo porta a pensare continuamente ad «assegnare variabili» e a quali variabili modificare
 - in quale variabile metto questo elemento?
 - in quale variabile metto il risultato di questa operazione?
- Dobbiamo invece chiederci «quali trasformazioni fare»
 - per ottenere il risultato, cosa mi serve?
 - come lo ottengo dal/dagli elemento/i che ho?
 - *pensa all'algoritmo!*



UN PRIMO ESEMPIO (1/5)

- Data una lista di Persone
 - si vuole una stringa-risultato ottenuta concatenando le **toString** delle varie istanze

APPROCCIO CLASSICO

- scorrere gli elementi della lista → ciclo **for** o **foreach**
- per ogni elemento, invocare **toString**
(mettendo il risultato in una variabile temporanea di appoggio)
- concatenare la stringa così ottenuta alle precedenti
(serve un accumulatore per il risultato)
- alla fine, il risultato è nell'accumulatore utilizzato



UN PRIMO ESEMPIO (2/5)

- Data una lista di Persone
 - si vuole una stringa-risultato ottenuta concatenando le **toString** delle varie istanze

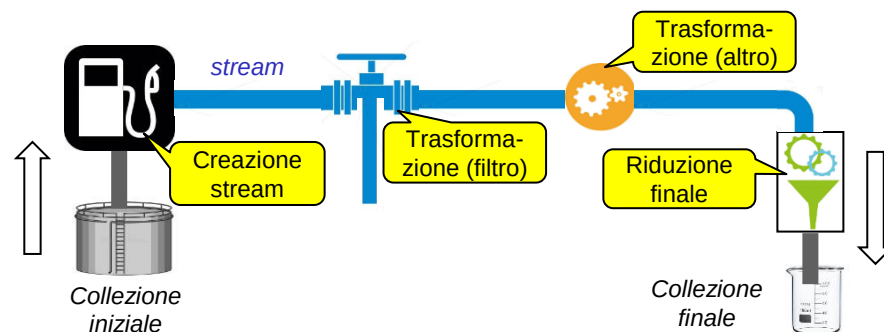
```
String res = "";  
for (Persona p : lista){  
    res += p.toString();  
}
```

APPROCCIO CLASSICO

- scorrere gli elementi della lista → ciclo **for** o **foreach**
- per ogni elemento, invocare **toString**
(mettendo il risultato in una variabile temporanea di appoggio)
- concatenare la stringa così ottenuta alle precedenti
(serve un accumulatore per il risultato)
- alla fine, il risultato è nell'accumulatore utilizzato

UN PRIMO ESEMPIO (3/5)

- Data una lista di Persone
 - si vuole una stringa-risultato ottenuta concatenando le **toString** delle varie istanze

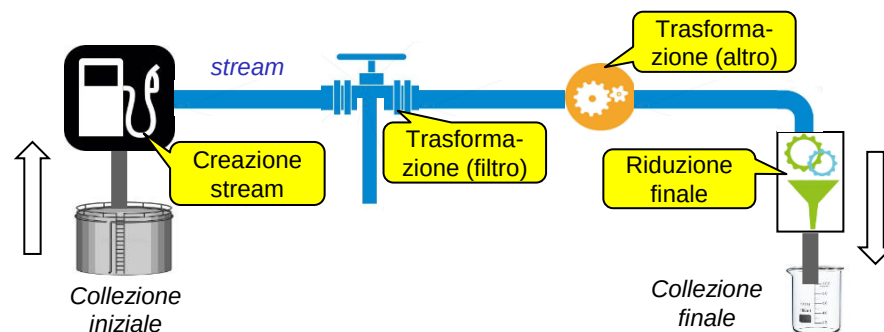


APPROCCIO A STREAM

- si **crea** dalla lista uno **stream di persone** → **stream()**
→ ora nello stream fluiscono una ad una le persone
- si **trasforma** ogni persona nel **risultato-stringa** → **map(...)**
→ ora nello stream fluiscono le corrispondenti stringhe
- si **accumulano** tali stringhe → **collect(...)**
in un *collettore appropriato* → **Collectors.joining(...)**

UN PRIMO ESEMPIO (4/5)

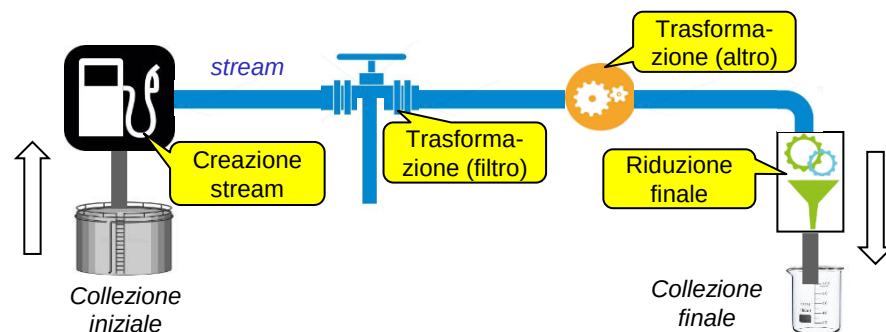
- Data una lista di Persone
 - si vuole una stringa-risultato ottenuta concatenando le `toString` delle varie istanze



```
String res =  
  lista.stream() Creazione stream  
    .map(Persona::toString) Trasformazione da Persona a Stringa  
    .collect(Collectors.joining("\n")) ; Riduzione finale (concatenamento con join)
```

UN PRIMO ESEMPIO (5/5)

- Variante: solo maggiorenni
 - si vuole una stringa-risultato ottenuta concatenando le `toString` delle **sole** istanze che soddisfano la condizione



```
String res =
  lista.stream()
    .filter(p -> p.getEta() >= 18)
    .map(Persona::toString)
    .collect(Collectors.joining("\n"));
```

Creazione stream

Trasformazione (filtro)
Solo le persone che...

Trasformazione da Persona a Stringa

Riduzione finale
(concatenamento con join)

Stream: funzionamento pratico



FASE 1: CREAZIONE STREAM

- Uno stream *di oggetti*, **Stream<Qualcosa>**, viene creato:
 - dal **metodo stream()** invocato su qualsiasi collection
 - dal **factory method Stream.of(array)** nel caso di array
 - da altri factory methods di altre classi (es. **Random**)
- PROBLEMA: i tipi primitivi in Java non sono classi ☹
→ l'approccio sopra *non funziona* per loro!
- Possibili soluzioni:
 - usare stream dei corrispondenti *tipi wrapper*
→ **Stream<Integer>**, **Stream<Float>**, etc.
 - *predisporre tipi di stream ad hoc* almeno per i *tipi primitivi più usati*
→ **IntStream**, **LongStream**, **DoubleStream**



FASE 1: CREAZIONE STREAM

- Uno stream *di oggetti*, `Stream<Qualcosa>`, viene creato:
 - dal **metodo `stream()`** invocato su qualsiasi collection
 - dal **factory method `Stream.of(array)`** nel caso di array
 - da altri factory methods di altre classi (es. `Random`)
- ESEMPI

```
List<String> elencoParole = List.of(...);  
Stream<String> st = elencoParole.stream();
```

Stream di
stringhe

```
String[] arrayParole = {...};  
Stream<String> st = Stream.of(arrayParole);
```

```
int[] valori = {7,21,4,-5,6,-6,18,22,-100,21};  
IntStream st = IntStream.of(valori);
```

Stream di int



FASE 1: CREAZIONE STREAM

- Uno stream può essere *generato elemento per elemento*
 - da **Stream.generate**, **Stream.iterate**
 - da altri factory methods di altre classi (es. **Random**)
- Il modello di esecuzione *lazy* consente di generare anche uno stream *potenzialmente infinito*
 - ESEMPIO: generare iterativamente tutti i numeri pari, da 2 in poi
 - se gli elementi fossero prodotti tutti all'inizio, sarebbe un ciclo infinito
 - *con la lazy evaluation no*, perché sono prodotti via via, solo *al bisogno*: lo stream predispone la «machinery», ma emetterà gli elementi solo quando gli saranno effettivamente richiesti

```
Stream<Double> s = Stream.generate(Math::random) ;
```

```
Stream<Integer> pari = Stream.iterate(2, n->n+2) ;
```



FASE 2: OPERARE SULLO STREAM

- Una volta creato, sullo stream si invocano *operazioni*
 - le trasformazioni intermedie *lavorano sui dati*
 - filtrano, separano, modificano – tipicamente, passando la specifica operazione da compiere sotto forma di *lambda*
 - restituiscono sempre un altro `Stream<Qualcosa>` su cui agiscono le «stazioni» successive
 - ESEMPIO: filtrare tutte le parole di lunghezza maggiore di 8
 - si crea lo stream dalla lista di stringhe → è uno stream di stringhe
 - la «stazione» `filter` fa passare solo le stringhe «gradite»
→ alla sua uscita c'è un altro stream, sempre di stringhe
 - per sintetizzare il risultato dovremo poi scegliere il «giusto» riduttore

```
Stream<String> st = elencoParole.stream();  
long n = st.filter( p->p.length()>8 ).riduzione;
```



FASE 2: OPERARE SULLO STREAM

- Una volta creato, sullo stream si invocano *operazioni*
 - le trasformazioni intermedie *lavorano sui dati*
 - filtrano, separano, modificano – tipicamente, passando la specifica operazione da compiere sotto forma di *lambda*
 - restituiscono sempre un altro `Stream<Qualcosa>` su cui agiscono le «stazioni» successive
 - ESEMPIO: trasformare tutte le parole in maiuscolo
 - la «stazione» `map` trasforma ogni stringa in un'altra
→ alla sua uscita c'è un altro stream, sempre di stringhe
 - per sintetizzare il risultato dovremo poi scegliere il «giusto» riduttore

```
Stream<String> st = elencoParole.stream();  
List<String> paroleMaiuscole=  
    st.map( p->p.toUpperCase() ).riduzione;
```



FASE 3: SINTESI DEL RISULTATO

- Per sintetizzare il risultato finale, il *riduttore* può:
 - **limitarsi ad accumulare** in una collection i dati che arrivano
→ metodo **collect** (con opportuno *collettore*)
 - **elaborare** i dati che arrivano (es. contarli, raggrupparli, etc.)
→ metodo di riduzione specifico
 - ESEMPLI:
 - per *contare* le parole filtrate (di lunghezza>8) *non interessa accumularle*:
basta un riduttore che le conti (e nel mentre le butti) → **count**
 - per *accumulare* tutte le parole maiuscole dovremo invece collezionarle in una nuova collection → **collect + opportuno collettore**

```
long n = st.filter( p->p.length()>8 ).count();  
List<String> paroleMaiuscole=  
    st.map( p->p.toUpperCase() )  
      .collect(Collectors.toList());
```

Stream alimentati da array



STREAM & ARRAY: CREAZIONE

- In fase 1, tipicamente, uno stream viene creato a partire da una collection, tramite il metodo `stream()` di `Collection`

```
List<String> elencoParole = ...;  
Stream<String> st = elencoParole.stream();
```

- Gli array, però, *non* sono collection e perciò *non* possiedono tale metodo: per creare uno stream si usa il **factory method** **`Stream.of(array)`**

```
String[] array = { "ciao", "ragazzi" };  
Stream<String> st = Stream.of(array);
```

- lo stesso vale per i tre **stream di tipi primitivi** con i relativi factory methods `IntStream.of(array)`, `LongStream.of(array)`, `DoubleStream.of(array)`



STREAM & ARRAY: RIDUZIONE

- In fase 3, per **accumulare i risultati in una collection** si usa il riduttore **collect (...)**
 - l'argomento è un opportuno **Collector**
 - la **libreria Collectors** definisce metodi per produrre collettori adatti a ogni struttura dati: **toList()**, **toSet()**, **toMap(...)**
- Se però si vuole **accumulare il risultato in un array**, occorre usare un riduttore ad hoc: **toArray**
 - MOTIVO: l'array è l'unica struttura tipizzata *fin nella JVM*, mentre tutte le altre collection sono tipizzate **<T>** solo a livello di codice → a differenza degli altri, ha bisogno di **sapere il tipo di array** da produrre
 - **l'argomento è una method reference al costruttore di quel tipo-array**
 - l'argomento *non è previsto* invece nel caso dei tre *stream di tipi primitivi*, poiché in tal caso il tipo dell'array da produrre è ovvio

COLLETTORI

- Per collezionare i risultati in una lista o set:

```
st.collect(Collectors.toList())
```

```
st.collect(Collectors.toSet())
```

- Per collezionare i risultati in una mappa:

```
st.collect(Collectors.toMap(keyMapper, valueMapper))
```

dove *keyMapper* è una funzione che produce le chiavi e *valueMapper* è una funzione che produce i valori

- Per collezionare i risultati in un array:

```
st.toArray(Type[]::new)
```

dove *Type* è il tipo dell'array desiderato

NB: per i tre stream di tipi primitivi, semplicemente `st.toArray()` perché il tipo dell'array risultato è già noto a priori



ESEMPIO 1

- Operiamo su uno stream di **Persona**
 - estraiamo i *nomi* delle persone → diventa uno stream di **String**
- Vogliamo mettere tali risultati in un nuovo *array di stringhe*
 - usiamo quindi il riduttore **toArray(String[] :: new)**

```
record Persona(String nome, LocalDate dataNascita){}

Stream<Persona> st = Stream.of( new Persona[]{
    new Persona("Tom", LocalDate.of(2020,3,2)),
    new Persona("Ann", LocalDate.of(2006,2,9)),
    new Persona("Sam", LocalDate.of(2001,7,8)),
    new Persona("Tom", LocalDate.of(2002,5,1))} );

String[] nomi = st.map(Persona::nome)
                  .toArray(String[] :: new);

System.out.println(Arrays.toString(nomi));
```

```
[Tom, Ann, Sam, Tom]
```



ESEMPIO 1 – VARIANTE

- Come sopra, ma vogliamo i nomi *distinti*, *senza ripetizioni*
 - aggiungiamo il filtro **distinct**
 - OSSERVA: nel modello imperativo, avremmo dovuto costruirci a mano tutto un complesso algoritmo di ricerca duplicati! Qui invece... 😊

```
Stream<Persona> st = Stream.of( new Persona[] {  
    new Persona("Tom", LocalDate.of(2020,3,2)),  
    new Persona("Ann", LocalDate.of(2006,2,9)),  
    new Persona("Sam", LocalDate.of(2001,7,8)),  
    new Persona("Tom", LocalDate.of(2002,5,1)) } );  
String[] nomi = st.map(Persona::nome)  
                  .distinct()  
                  .toArray(String[]::new);  
System.out.println(Arrays.toString(nomi));
```

```
[Tom, Ann, Sam]
```

ESEMPIO 2

- Operiamo su uno stream di **Integer** (*non di int*)
 - filtriamo solo i valori positivi (unboxing automatico)
- Vogliamo mettere i risultati in un *nuovo array di Integer*
 - usiamo quindi il riduttore **toArray(Integer[] :: new)**

```
Stream<Integer> st =  
    Stream.of(new Integer[]{3, 67, 1, 18, -2, 9});  
  
Integer[] result =  
    st .filter(x -> x > 0)  
      .toArray(Integer[] :: new);  
  
System.out.println(Arrays.toString(result));
```

Unboxing
Integer → int

```
[3, 67, 1, 18, 9]
```



ESEMPIO 2 – VARIANTE

- Operiamo invece su uno **stream di valori primitivi `int`**
 - filtriamo solo i valori positivi
- Vogliamo mettere i risultati in un *nuovo array di `int`*
 - usiamo quindi il riduttore **`toArray()`**

```
IntStream st =  
    IntStream.of(new int[]{3,67,1,18,-2,9});  
  
int[] result = st.filter(x -> x>0)  
    .toArray();  
  
System.out.println(Arrays.toString(result));
```

Riduttore senza
constructor reference

```
[3, 67, 1, 18, 9]
```

Galleria delle trasformazioni più usate



TRASFORMAZIONI FONDAMENTALI

- Ci sono tre trasformazioni fondamentali (con più varianti):
 - **filter** (*Predicate*): filtra nel nuovo stream-risultato solo gli elementi dello stream che soddisfano la condizione, ossia solo quelli per cui il predicato è vero
 - **map** (*Function*) [e variante **mapToObj** (*Function*)]: applica a ogni elemento x dello stream la funzione data: $y = f(x)$, producendo un nuovo stream con i rispettivi risultati y (di tipo diverso dal primo). La variante **mapToObj** si usa per operare su stream di *tipi primitivi*, producendo come risultato un nuovo stream di *tipi-oggetto*.
 - **flatMap** (*Function*): se la funzione *Function* produce come risultato uno stream (non un singolo elemento), la trasformazione **map** darebbe come risultato uno *stream di stream*: **flatMap** li unisce in un unico stream, «appiattendo» il risultato

ALTRE TRASFORMAZIONI

- Si possono anche **mettere vincoli sul numero di elementi**:
 - **limit(n)** lascia passare solo i primi n elementi (gli altri li butta)
→ *cruciale per limitare gli stream potenzialmente infiniti*
 - **skip(n)** fa l'opposto: esclude i primi n elementi, fa passare gli altri

ESEMPIO

```
Stream<Double> st1 =  
    Stream.generate(Math::random)  
        .limit(10)  
        .skip(5);
```

Stream di Double
potenzialmente infinito

Limitando ai primi 10 item, il
generatore random viene
sollecitato solo 10 volte
→ non c'è alcun ciclo infinito

Poi però i primi 5 item
vengono scartati

Il risultato è ancora uno stream di
numeri, da elaborare successivamente



RIDUZIONI FONDAMENTALI

- Riduzioni fondamentali:
 - **count**, **min**, **max**, **average**: collettori che *operano sugli elementi senza accumularli*: rispettivamente conta gli elementi, trova il min, il max, la media... etc.
 - **collect (collector)**: è il riduttore-accumulatore, che *accumula gli elementi* in un collettore (tipicamente, un array o una collection)
 - **reduce (function)**: è il riduttore general-purpose, che applica a ogni elemento dello stream la funzione data: *è tale funzione che specifica cosa fare del risultato* [disponibile in due varianti]
 - **forEach (operation)**: riduttore che effettua un'iterazione interna, ripetendo l'operazione data su tutti gli elementi dello stream



COLLETTORI FONDAMENTALI

- Collettori più comuni da usare insieme a `collect`:

- `toList`, `toSet`, `toMap(fk, fv)`
- `toCollection(constructorRef)`

Questo è il collettore più generale, che può accumulare in una qualsiasi collection, anche se per essa non esiste il collettore specifico (caso d'uso tipico: per accumulare in un `TreeSet` o `TreeMap`)

ESEMPIO

Per accumulare in un `TreeSet` (concretizzazione di `SortedSet`), si può scrivere `toCollection(TreeSet::new)`



COLLETTORI FONDAMENTALI

- Collettori «raggruppanti»:
 - **groupBy(criterio)**, **partitioningBy(criterio)**: riuniscono gli elementi "a gruppi" secondo il criterio dato; il risultato è *una mappa* con tante entry quanti i gruppi, e per ogni riga la *lista* degli elementi dello stesso gruppo (esistono varianti per altre collection)
 - **summarizingXX**, **summingXX**, **averagingXX** (XX = Int, Long, Double) anche questi riuniscono gli elementi "a gruppi", ma il criterio «è nel nome» del collettore: il risultato è *una mappa* con tante entry quanti i gruppi e come valore il risultato (somma, media, etc.) del gruppo
- Solo per stream di stringhe:
 - **joining**: *restituisce una stringa* ottenuta concatenando, col separatore indicato, tutte le stringhe dello stream

Stream di tipi primitivi



STREAM DI TIPI PRIMITIVI

- Gli stream *di tipi primitivi* `int`, `long`, `double` sono creati:
 - dai **factory method** `IntStream.of(...)` nel caso di `int`
 - dai **factory method** `LongStream.of(...)` nel caso di `long`
 - dai **factory method** `DoubleStream.of(...)` nel caso `double`
- Possono essere anche creati da **metodi generativi**
 - es: `generate`, `iterate`, `range`, `rangeClosed`, `concat`
 - a volte da metodi di classi come `Random` (e altre)

ESEMPI

```
IntStream is = IntStream.iterate(3, x -> 2*x);
```

```
DoubleStream ds = DoubleStream.generate(  
    Math::random);
```

```
IntStream is = new Random().ints(0,10);
```



STREAM DI TIPI PRIMITIVI

- NON è possibile convertire stream di tipi primitivi in stream di oggetti (wrapper) tramite boxing/unboxing automatici
 - per convertire uno **stream di tipi primitivi** in uno **stream di oggetti wrapper** si usa il metodo **boxed()**

```
IntStream is = new Random().ints(0,M);  
Stream<Integer> iis = is.boxed();
```

- per convertire uno **stream di tipi primitivi in uno stream di oggetti qualsiasi** si usa il metodo **mapToObj (mapper)**, dove *mapper* trasforma un valore primitivo nell'oggetto desiderato

```
IntStream is = new Random().ints(0,M);  
Stream<Frazione> fis = is.mapToObj (mapper);
```

In questo caso, *mapper* mappa **int** in **Frazioni**



STREAM DI TIPI PRIMITIVI: ESEMPI

- ESEMPIO: sorteggiare K frazioni da una lista

```
List<Frazione> frazioni = List.of(...);
```

```
new Random()
```

```
.ints(0, frazioni.size())
```

Genera un IntStream di valori compresi fra 0 (incluso) e size (escluso)

```
.distinct()
```

Impone che siano tutti distinti

```
.limit(4)
```

Limita/chiede 4 valori (implica size=>4, altrimenti endless loop!)

```
.mapToObj( n -> frazioni.get(n) )
```

```
.forEach(System.out::println);
```

Stampa le frazioni che
via via arrivano

Dato un intero, lo usa come indice per
recuperare una frazione dalla lista

Stream paralleli

STREAM PARALLELI (1/2)

- Con gli stream paralleli è possibile *parallelizzare il codice senza sforzo*
 - o si *crea lo stream già parallelo*, con `parallelStream()`
 - oppure si *rende parallelo* uno stream già esistente, con `parallel()`

```
List<String> elencoParole = ...;  
long num = elencoParole.stream()  
    .filter( p -> p.length()>8 )  
    .count();
```

Stream sequenziale

```
List<String> elencoParole = ...;  
long num = elencoParole.parallelStream()  
    .filter( p -> p.length()>8 )  
    .count();
```

Stream parallelo

STREAM PARALLELI (2/2)

- Con gli stream paralleli è possibile *parallelizzare il codice senza sforzo*
 - o si *crea lo stream già parallelo*, con `parallelStream()`
 - oppure si *rende parallelo* uno stream già esistente, con `parallel()`

```
List<Integer> list = new Random()
    .ints(1,100)
    .parallel()
    .limit(500)
    .mapToObj(i->i*i)
    .collect(Collectors.toList());
```

Stream inizialmente sequenziale...

... trasformato in stream parallelo

STREAM PARALLELI: GUADAGNO

- Ci si guadagna davvero? Facciamo una prova!
 - generiamo 500 interi ed elaboriamoli sia con stream sequenziale, sia con stream parallelo (senza/con chiamata `.parallel()`)
 - e... *misuriamo i tempi!*

```
long t0 = System.currentTimeMillis();  
List<Integer> list1 = new Random().ints(1,100).parallel()  
    .limit(500)  
    .mapToObj(i->i*i)  
    .collect(Collectors.toList());  
long t1= System.currentTimeMillis();  
System.out.println(t1-t0);
```

Limita lo stream (potenzialmente infinito) ai primi 500 valori

Calcola i quadrati

Mette i risultati (degli Integer) in una lista

35	32	31
12	10	8

Lo stream parallelo guadagna
circa un fattore 3
(in questo caso, su questo pc)

Galleria di esempi

ESEMPI (1)

```
List<String> elencoParole = ...;
```

```
int numeroParole =
```

```
    elencoParole.stream().filter(p -> p.length()>4)
```

```
    .count();
```

Trasformazione

Predicato

Riduzione

```
List<String> elencoParole = ...;
```

```
var listaParoleMaiuscole =
```

```
    elencoParole.stream().map( p -> p.toUpperCase())
```

```
    .collect(Collectors.toList());
```

Trasformazione

Funzione

Riduzione

ESEMPI (2)

```
Stream<List<Integer>> st = Stream.of(  
    Arrays.asList(1,2),  
    Arrays.asList(3,4),  
    Arrays.asList(5)  
);
```

Tre liste di interi
[1,2], [3,4], [5]

Stream di liste di interi
[[1,2], [3,4], [5]]

```
Stream<Integer> stf =  
    st.flatMap( list -> list.stream() );
```

Stream di interi "appiattito"
[1, 2, 3, 4, 5]

Produce uno stream per ogni lista dello stream originale **st**,
quindi in questo momento ha uno stream di tre stream:
flatMap li appiattisce in un unico stream

ESEMPI (3)

Stampa base

```
Stream<?> st = ...;  
st.forEach( System.out::println );
```

Stampa un po' più evoluta:

```
Stream<?> st = ...;  
st.forEach( e -> System.out.print(e +", ") );  
System.out.println();
```

Problema: *il forEach consuma lo stream*

- MOTIVO: è una *riduzione*, ossia una *operazione terminale*
- dunque, a iterazione terminata lo stream è chiuso
→ *non è più possibile svolgere su di esso alcuna operazione*

ESEMPI (4)

Alternativa: l'operazione **peek** che *duplica lo stream*

- si itera sugli elementi di un *nuovo stream duplicato*
- lo stream originale non viene toccato

```
st.peek( e -> System.out.print(e +", ") )  
  .doSomethingElseOnOriginalStream(..);
```

Esempio completo:

```
Double[] numeri =  
  Stream.iterate(1.0, x -> 2*x)  
    .peek(System.out::println)  
    .limit(10)  
    .toArray(Double[]::new);
```

- Genera le potenze di 2
- Lo stream duplicato le stampa
- Lo stream principale mette in un array di Double le prime 10

ESEMPI (5)

Il metodo **reduce** esprime la generica riduzione

- 1° argomento: *l'elemento neutro dell'operazione*
- 2° argomento: *l'operazione associativa da svolgere*
- applica ripetutamente l'operazione a coppie di elementi consecutivi

ESEMPIO: somma degli elementi di uno stream di **Integer**

```
Stream<Integer> st
    = Stream.of( new Integer[] {1,2,54,23,12,-4,22} );
Integer sum = st.peek( System.out::println)
                .reduce(0, (res,v) -> res+v );
```

Elemento neutro della somma

Risultato: 110

Operazione da applicare (boxing/unboxing!)

$\text{Res}_1 = 0 + v_1$, $\text{Res}_2 = \text{Res}_1 + v_2$, ...

$\text{Res}_N = \text{Res}_{N-1} + v_N$

ESEMPI (5)

ESEMPIO: variante con `IntStream`

```
IntStream st  
  = Stream.of( new int[] {1,2,54,23,12,-4,22} );  
int sum = st.peek( System.out::println)  
            .reduce(0, (res,v) -> res+v );
```

Elemento neutro della somma

Risultato: 110

Operazione da applicare (no boxing):

$\text{Res}_1 = 0 + v_1$, $\text{Res}_2 = \text{Res}_1 + v_2$, ..

$\text{Res}_N = \text{Res}_{N-1} + v_N$



ESEMPI (6)

Esiste anche una versione di **reduce a singolo argomento**

- l'unico argomento è la funzione: non c'è l'elemento neutro iniziale
- *MA il risultato potrebbe non esistere* se lo stream fosse vuoto

Per questo, essa restituisce un **Optional**

- appunto perché il risultato potrebbe non esistere
- occorre quindi verificare che il risultato non sia **empty** ed estrarre eventualmente il valore

ESEMPI (7)

ESEMPIO: ulteriore variante con **reduce** a singolo argomento:

```
IntStream st1
    = Stream.of( new int[]{1,2,54,23,12,-4,22} );
IntStream st2 = Stream.of( new int[]{} );
OptionalInt sum1 = st1.reduce( (res,v) -> res+v );
OptionalInt sum2 = st2.reduce( (res,v) -> res+v );
System.out.println("Somma : " +
    (sum1.isPresent() ? sum1.getAsInt() : "indefinita"));
System.out.println("Somma : " +
    (sum2.isPresent() ? sum2.getAsInt() : "indefinita"));
```

Vuoto

empty

ESEMPI (8)

ESEMPIO: somma *delle lunghezze delle stringhe* di uno stream

- sfruttiamo **mapToInt** per mappare ogni stringa nell'intero che ne rappresenta la lunghezza

```
String[] array = { "ciao", "bravi", "ragazzi" };  
int sumLengths = Stream.of(array)  
    .mapToInt( String::length )  
    .sum();
```

Mappa ogni parola
in un intero..

..che è la sua
lunghezza:

La riduzione finale li
somma tutti

Risultato: 16



ESEMPI (9)

Un'altra riduzione utili (fra le tante) è **average**

- fa la media e restituisce un `OptionalDouble`

```
double media =  
    Stream.of( new int[]{1,2,54,23,12,-4,22} )  
        .average()  
        .getAsDouble();
```

Risultato: 17.7

ESEMPI (10)

ESEMPIO: fare la media di un insieme di interi *che però non sono già disponibili, ma devono essere prima calcolati*

CASO CONCRETO: *media delle lunghezze di uno stream di parole*

- Il **collettore averagingInt** fa esattamente al caso nostro
 - *il suo argomento è l'operazione da svolgere*
 - *qui, ricavare la lunghezza della stringa corrente*
- Alla fine, calcola la media (reale) di un insieme di valori interi

```
String[] array = { "ciao", "bravi", "ragazzi" };  
Stream<String> stP = Stream.of(array);  
double average =  
    stP.collect(Collectors.averagingInt(String::length));
```

Risultato: 5.333

Ricavo int da string

ESEMPI (11)

ESEMPIO: associare a ogni codice fiscale il nome della persona

- il **collettore toMap** fa esattamente al caso nostro
- keyMapper = estrattore codice fiscale
- valueMapper = estrattore nome

```
Persona[] arrayPersone = {  
    new Persona("Mario Rossi", "RSSMRA76H12A944I"),  
    new Persona("Lucia Verdi", "VRDLCU98T65H223X") };  
  
List<Persona> listaPersone = Arrays.asList(arrayPersone);  
Stream<Persona> st = listaPersone.stream();  
  
Map<String, String> mappa1 =  
    st.collect( Collectors.toMap(Persona::getCodFisc,  
                                Persona::getNome) );
```

Estrattori

```
{ RSSMRA76H12A944I=Mario Rossi, VRDLCU98T65H223X=Lucia Verdi }
```



ESEMPI (12)

VARIANTE: associare a ogni codice fiscale *l'oggetto persona*

- Il **collettore toMap** fa ancora al caso nostro
- valueMapper deve restituire «la persona stessa»... come fa?
 - si potrebbe scrivere semplicemente **f -> f**
ma sarebbe sgradevole, è una funzione «degenerare»
 - più elegante usare il metodo **Function.identity**

```
Map<String, Persona> mappa2 =  
    st.collect( Collectors.toMap(Persona::getCodFisc,  
                                   Function.identity() ) );
```

```
{ RSMRA76H12A944I=Persona@41629346,  
  VRDLCU98T65H223X=Persona@404b9385 }
```

ESEMPI (13)

Il metodo **groupBy** raggruppa gli elementi

- l'argomento è un classificatore (*Classifier*) ossia una funzione che produce risultati di diverse «categorie»
- viene generata una mappa avente per chiave il gruppo e per valore la lista degli elementi di quel gruppo

ESEMPIO: *raggruppare le frazioni per denominatore*

```
List<Frazione> frazioni = List.of(...);  
  
var myMap = frazioni  
    .stream()  
    .distinct()  
    .collect( Collectors.groupBy( f -> f.den() ) );
```

```
{3=[2/3], 4=[1/4], 5=[-2/5, 3/5], 7=[2/7]}
```

ESEMPI (14)

ESEMPIO: raggruppare le culture locali per paese

```
Stream<Locale> locales =  
    Stream.of( Locale.getAvailableLocales() );  
  
Map<String, List<Locale>> mappaPaeseLocales =  
    locales.collect(  
        Collectors.groupingBy( Locale::getCountry ) );
```

Output:

```
[it_IT]  
[fr_CA, en_CA]  
[fr_CH, de_CH, it_CH]
```

```
System.out.println( mappaPaeseLocales.get("IT") );  
System.out.println( mappaPaeseLocales.get("CA") );  
System.out.println( mappaPaeseLocales.get("CH") );
```



ESEMPI (15)

Nel caso specifico di **due** gruppi (partizione), è opportuno preferire il metodo **partitioningBy**

- l'argomento è un *classificatore boolean*

ESEMPIO: *separare le frazioni negative da quelle positive*

```
List<Frazione> frazioni = List.of(...);  
var myMap = frazioni  
    .stream()  
    .distinct()  
    .collect( Collectors.partitioningBy(  
        f -> f.den() * f.num() >= 0 ) );
```

```
{false=[-2/5], true=[2/3, 1/4, 2/7, 3/5]}
```

ESEMPI (16)

Nel caso specifico di **due** gruppi (partizione), è opportuno preferire il metodo **partitioningBy**

- l'argomento è un *classificatore boolean*

ESEMPIO: *separare le culture locali fra chi parla inglese e chi no*

```
Stream<Locale> locales =  
    Stream.of( Locale.getAvailableLocales() );  
  
Map<Boolean, List<Locale>> mappaEnglish =  
    locales.collect( Collectors.partitioningBy(  
        loc -> loc.getLanguage().equals("en") ) );
```

```
[en_US, en_SG, en_MT, en, en_PH, en_NZ, en_ZA, en_AU, en_IE,  
en_CA, en_IN, en_GB] Elementi spuri  
[ar_AE, ar_JO, ar_SY, hr_HR, fr_BE, es_PA, mt_MT, es_VE, ... ]
```

Gli elementi spuri sono lingue non istanziate su uno specifico paese

ESEMPI (17)

- Di default, `groupingBy` e `partitioningBy` raggruppano gli elementi in *liste*
- Si possono ottenere *strutture dati diverse* fornendo un *collettore specifico* come ulteriore argomento

ESEMPIO: raggruppare le culture locali per paese in Set

```
Stream<Locale> locales =  
    Stream.of( Locale.getAvailableLocales() );  
  
Map<String, Set<Locale>> mappaPaeseLocales =  
    locales.collect(  
        Collectors.groupingBy( Locale::getCountry,  
                                Collectors.toSet() ) );
```

ESEMPI (18)

Più in generale, `groupBy` e `partitioningBy` supportano il *downstream processing*, ossia la possibilità di *ulteriori elaborazioni sui dati già raggruppati*

ESEMPIO: contare le culture locali raggruppate

```
Stream<Locale> locales =  
    Stream.of( Locale.getAvailableLocales() );  
  
Map<String, Long> mappaPaeseLocales =  
    locales.collect(  
        Collectors.groupingBy( Locale::getCountry,  
                                Collectors.counting() ) );
```

```
1 System.out.println( mappaPaeseLocales.get("IT") );  
2 System.out.println( mappaPaeseLocales.get("CA") );  
3 System.out.println( mappaPaeseLocales.get("CH") );
```

ESEMPI (19)

VARIANTE: paesi che parlano inglese o meno

- senza i codici linguaggio generici

```
Stream<Locale> locales =  
    Stream.of( Locale.getAvailableLocales() );  
Map<Boolean, Long> mappaContaEnglish =  
    locales.collect(  
        Collectors.partitioningBy(  
            loc -> loc.getLanguage().equals("en")  
                && !loc.getCountry().equals(""),  
            Collectors.counting() ) );
```

```
11      System.out.println( mappaContaEnglish.get(true) );  
149     System.out.println( mappaContaEnglish.get(false) );
```

Esempi da compiti d'esame

(sempre utili.... 😊)



I SEMPRE UTILI (1/5)

Problema: data una lista di elementi, produrre un *set* (senza duplicati) dei soli elementi tali che...

```
Set<Elemento> scelti =  
    lista.stream().filter(e -> taleChe(e)).collect(Collectors.toSet());
```

Problema: data una lista di elementi, produrre un'altra *lista senza duplicati* dei soli elementi tali che...

```
List<Elemento> result =  
    lista.stream().filter(e -> taleChe(e)).distinct().collect(  
                                                (Collectors.toList());
```



I SEMPRE UTILI (2/5)

Problema: produrre una *lista di N valori casuali* compresi fra 0 e M (possono esserci duplicati)

```
List<Integer> elementiCasuali =  
    new Random().ints(0,M).limit(N).boxed().collect(Collectors.toList());
```

Perché lo stream non sia infinito

Trasforma in `IntStream` in uno `Stream<Integer>`
(e più in generale uno stream di tipi primitivi in uno di tipi wrapper)

Idem con valori *distinti*:

```
List<Integer> scelti =  
    new Random().ints(0,M).distinct().limit(N).boxed().collect(...);
```

Idem con un *set* (elimina i duplicati, quindi sono *al più* N valori)

```
Set<Integer> scelti =  
    new Random().ints(0,M).limit(N).boxed().collect(Collectors.toSet());
```



I SEMPRE UTILI (3/5)

Problema: produrre una *lista di (non più di N) elementi selezionati casualmente* da una lista di M elementi possibili (di tipo Element)

- si genera una lista di N interi casuali (con o senza duplicati) fra 0 e M
- si sfruttano tali indici per estrarre elementi **dall'altra lista (source)**
- si accumulano nella lista-risultato solo gli elementi sorteggiati

```
List<Elemento> scelti =  
    new Random().ints(0,M).distinct().limit(N)  
        .mapToObj(source::get)  
        .collect(Collectors.toList());
```

Come caso particolare, convertire lo stream di `int` in stream di `Integer` tramite `mapToObj` anziché `boxed`:

```
List<Integer> elementiCasuali =  
    new Random().ints(0,M).limit(N).mapToObj(i -> i).collect(...);
```



I SEMPRE UTILI (4/5)

Problema: data una *lista di documenti*

- caratterizzati ciascuno da identificativo univoco (ID) e timestamp

produrre la lista dei soli documenti *modificati entro quel timestamp*

```
List<Document> result =  
    lista.stream().filter( doc ->  
        doc.getTimestamp().isBefore(timestamp) ||  
        doc.getTimestamp().equals(timestamp) )  
    .collect(Collectors.toList());
```

Problema: come sopra, ma restituire *solo il documento più recente*

- anziché ammucciarli in lista, prendiamo solo il max
- il risultato è un Optional perché se la lista è vuota il max non esiste

```
Optional<Document> doc =  
    lista.stream().filter( doc ->  
        doc.getTimestamp().isBefore(timestamp) ||  
        doc.getTimestamp().equals(timestamp) )  
    .max(Comparator.comparing(Document::getTimestamp));
```



I SEMPRE UTILI (5/5)

Problema: data una *lista di tutti i voli aerei disponibili* (orario stagionale)

- caratterizzati da aeroporti e orari di partenza/arrivo + giorni in cui operano

produrre la lista dei soli voli *fra i due aeroporti dati* che siano *operanti in un ben preciso giorno della settimana* (dato anch'esso)

```
List<FlightSchedule> voliDisponibili =  
    orarioStagionale.stream().filter(fs ->  
        fs.getDepartureAirport().equals(departureAirport) &&  
        fs.getArrivalAirport().equals(arrivalAirport) &&  
        fs.getDaysOfWeek().contains(dayOfWeek) )  
    .collect(Collectors.toList());
```